

Celaya NFC 控制器 RTOS 软件开发

移植指南

文档版本 05

发布日期 2024-10-16

前言

概述

本文档详细的描述了如何把 NFC（Near Field Communication）控制器芯片适配到RTOS 的嵌入式系统中，主要包括简介和 NFC 系统架构、NFC 驱动移植方法、将NFC 控制器软件移植适配到 RTOS 的嵌入式系统中的具体方法、常见问题。

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

修改记录

文档版本	发布日期	修改记录
05	2024-10-16	- 更新“2.2 获取驱动源码”中的文件描述。 - 更新“2.4 编译适配”获取固件方式中相关文件描述。 - 更新“3.1 步骤一：获取 NFC 安装包”章节中的 HAL 层文件描述。 - 更新“3.2 步骤二：定制产品的 HAL 层接口函数”章节中示例代码文件和接口函数的名称。 - 更新“3.3 步骤三：把 HAL 层加入编译流”

文档版本	发布日期	修改记录
		程”章节中编译宏的命名、固件数组文件的名称。 - 新增“3.4 步骤四：把 Ndef 协议操作函数加入编译流程”章节。 - 新增“3.5 步骤五：添加编译宏”章节。 - 更新“A 缩略语”，添加 RW、CE 的缩略语说明
04	2023-02-06	- 更新全文的嵌入式系统环境描述，从 MCU 改为 RTOS - 更新“1 NFC 软件分层框图”小节内容描述。 - 更新“3.1 步骤一：获取 NFC 安装包”小节中软件目录和文件。 - 更新“3.3 步骤三：把 HAL 层加入编译流程”小节内容描述。 - 更新“2.3 代码移植和适配”中的驱动目录和文件描述等。 - 新增“安全函数库编译指导”
03	2022-08-23	- 更新“2.4 编译适配”小节内容描述。 - 更新“3.1 步骤一：获取 NFC 安装包”小节中增加 listen CE 相关文件。 - 更新“3.2 步骤二：定制产品的 HAL 层接口函数”小节中函数名称及内容描述。 - 更新“3.3 步骤三：把 HAL 层加入编译流程”小节中大数组方式固件加载的注意事项。 - 更新“3.7 步骤七：测试 NFC 功能”小节中固件存在 HOST 端的注意事项。
02	2022-06-23	- 新增“4.2 NFCEE 激活失败”小节。 - 新增“4.3 I2C 通路异常”小节。

文档版本	发布日期	修改记录
01	2022-04-29	第一次正式版本发布。

目 录

前言	i
1 NFC 软件分层框图	1
2 NFC 驱动	2
2.1 驱动描述	2
2.2 获取驱动源码	2
2.3 代码移植和适配	3
2.3.1 oal_os.h OS 接口	4
2.3.2 oal_plat_op.h 文件操作和日志打印接口	5
2.3.3 board_xxx.c 板级接口	6
2.3.4 安全函数库编译指导	7
2.4 编译适配	8
2.5 nfc_dev.h 上层访问驱动接口	8
3 NFC 软件移植	9
3.1 步骤一：获取 NFC 安装包	9
3.2 步骤二：定制产品的 HAL 层接口函数	11
3.3 步骤三：把 HAL 层加入编译流程	13
3.4 步骤四：把 Ndef 协议操作函数加入编译流程	13
3.5 步骤五：添加编译宏	13
3.6 步骤六：烧录版本	14
3.7 步骤七：测试 NFC 功能	14
4 常见问题	15
4.1 固件加载失败	15
4.2 NFCEE 激活失败	15

4.3 I2C 通路异常.....	16
A 缩略语.....	17

Draft

插图目录

图 1-1 RTOS 嵌入式系统上的 NFC 软件分层图	1
------------------------------------	---

表格目录

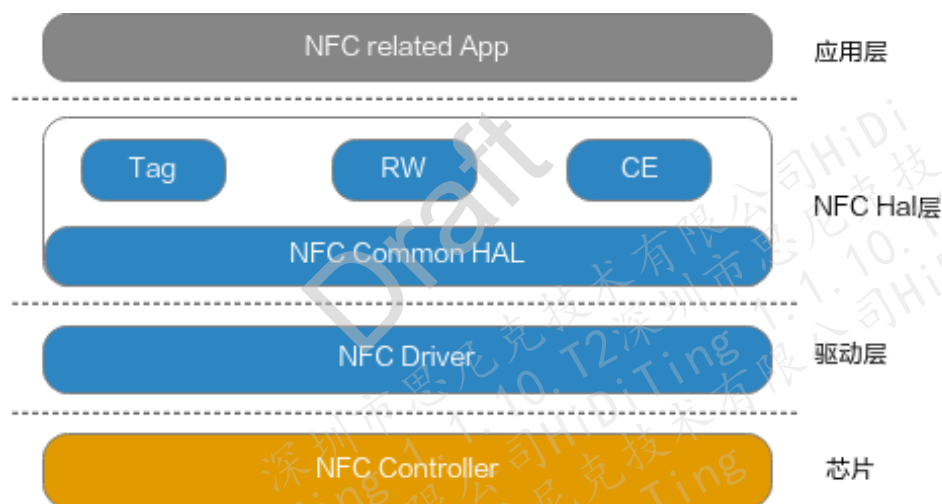
表 2-1 驱动代码说明	3
表 2-2 oal_os.h 接口函数说明	4
表 2-3 oal_plat_op.h 接口函数说明	5
表 2-4 board_xxx.c 接口函数说明	6
表 2-5 nfc_dev.h 接口函数说明	8
表 3-1 HAL 层定制化参数修改	11
表 3-2 RTOS 编译宏	13

1

NFC 软件分层框图

嵌入式系统上的 NFC 软件分层图如图 1-1 所示。

图1-1 RTOS 嵌入式系统上的 NFC 软件分层图



RTOS 嵌入式系统上的 NFC 软件分层图说明如下：

- 框图颜色说明：灰色为用户自有模块，蓝色为本方案软件，橙色为本方案硬件。
- NFC Driver 是 NFC 芯片的驱动，用于访问底层 NFC 控制器资源。
- NFC HAL 是硬件抽象层，用于向上层提供硬件服务的抽象接口。具体分为通用的 Common HAL 部分和功能模块 RW/CE/TAG 接口代码。
- 上层 NFC 相关的 APP 直接调用 HAL（Hardware Abstract Layer）层的接口处理业务。

2 NFC 驱动

2.1 驱动描述

2.2 获取驱动源码

2.3 代码移植和适配

2.4 编译适配

2.5 nfc_dev.h 上层访问驱动接口

2.1 驱动描述

NFC 驱动通过 I2C 和底层的 NFC 控制器通讯。主要实现 NFC 芯片固件加载、NFC 上电启动、给上层提供通用的接口用于访问底层 NFC 控制器资源。

2.2 获取驱动源码

获取 NFC 驱动安装包并解压。解压后 Vendor NFC 驱动相关文件如下：

- Makefile：驱动代码的 makefile
- comm_driver：驱动通用代码的子文件夹，具体文件请见安装包
- adapter：需要移植和适配的代码，具体文件请见安装包
- Kconfig：linux 系统适配相关，RTOS 环境请忽略

获取依赖的第三方库为“libboundscheck”，获取链接为

<https://gitee.com/openeuler/libboundscheck>。此库主要是提供 memcpy、strcpy 等内存操作库函数的安全版本。NFC 驱动代码使用了安全版本的内存操作函数。

2.3 代码移植和适配

NFC 驱动需要通过简单的移植和适配，嵌入 MCU 的环境才能编译运行。如上节介绍，驱动代码主要分为 comm_driver 模块和 adapter 模块，代码分别放在对应的目录。

- comm_driver 实现主要的驱动功能，不需要改动
- adapter 需要用户根据 MCU 环境适配实现固定的系统和板级接口，这些接口由 comm_driver 调用。adapter 中提供了 5 个接口用于上层访问 NFC 芯片，该接口用户可以定制改动，也可以使用默认配置。

代码结构与详细描述如表 2-1 所示。

表2-1 驱动代码说明

目录	子文件夹和文件的说明			
adapter(dir)	用户根据应用环境适配实现固定的系统和板级接口。			
-	oal(dir)	系统抽象层， 适配不同 OS 接口。		
	-	linux(dir)	linux 系统	
		-	oal_os.h	os 接口适配。
			oal_plat_op.h	文件操作接口适配。
		liteos(dir)	liteos 系统	
	-	oal_os.h	os 接口适配， 需要用户结合不同设备环境进行适配。	
		oal_plat_op.h	文件操作和日志打印接口适配， 需要用户结合不同设备环境进行适配。	
	product(dir)	产品板级接口适配。		
-	xxx(dir)	mcu 环境		
	-	board_xxx.c	实现 board_nfc.h 定义的板级接口， 这些接口被	

目录	子文件夹和文件的说明			
			comm_driver 模块调用， 需要用户结合不同设备环境进行适配。	
			nfc_dev.h	定义的 5 个接口用于上层访问 NFC 芯片，该接口用户可以定制改动，也可以使用默认配置。
			nfc_dev.c	
comm_driver(dir)	实现主要的驱动功能，不需要改动。			
-	board_nfc.h	-		
	priv_nfc_driver.h	-		
	priv_nfc_driver.c	-		

2.3.1 oal_os.h OS 接口

适配实现如表 2-2 所示接口，建议放在.h 文件中，通过 inline 或者函数宏实现，具体可以参考已有的代码实现。

表2-2 oal_os.h 接口函数说明

函数	功能说明
void oal_msleep(uint32_t time)	延时实现
void oal_udelay(uint32_t time)	
void oal_mutex_init(oal_mutex_t *lock)	互斥量实现
void oal_mutex_destroy(oal_mutex_t *lock)	
void oal_mutex_lock(oal_mutex_t *lock)	
oal_mutex_unlock(oal_mutex_t *lock)	
oal_spin_lock_init	临界段实现
oal_spin_lock_irqsave	

函数	功能说明
oal_spin_unlock_irqrestore	
void oal_init_waitqueue_head(oal_wait_queue_head_t *wq)	唤醒事件实现
int oal_wait_event_interruptible_timeout(oal_wait_queue_head_t wq, int condition, unsigned long timeout)	
void oal_wkup_event(oal_wait_queue_head_t wq)	

2.3.2 oal_plat_op.h 文件操作和日志打印接口

适配实现如表 2-3 文件操作接口，建议放在.h 文件中，通过 inline 或者函数宏实现，具体可以参考已有的代码实现。

表2-3 oal_plat_op.h 接口函数说明

函数	功能说明
oal_file_t *oal_file_open_readonly(const char *path)	只读打开文件
oal_file_read_ext(oal_file_t *file, oal_loff_t pos, char *buf, uint32_t count)	读文件
int32_t oal_file_close(oal_file_t *file)	关闭文件
int oal_file_open_get_len(const char *path, oal_file_t **fp, unsigned int *file_len)	获取文件长度
PS_PRINT_FUNCTION_NAME	打印当前函数名
PS_PRINT_DBG(s, args...)	debug 级别打印
PS_PRINT_INFO(s, args...)	info 级别打印
PS_PRINT_SUC(s, args...)	打印执行成功
PS_PRINT_WARNING(s, args...)	warning 级别打印
PS_PRINT_ERR(s, args...)	error 级别打印
PS_PRINT_ALERT(s, args...)	alert 级别打印

2.3.3 board_xxx.c 板级接口

主要实现 MCU 与 NFC 芯片之间的硬件接口操作，如 GPIO 控制、I2C 读写等。NFC 芯片的 I2C 地址是 0x28，配置 I2C 速率 full speed 400K，时钟占空比 40% ~ 60%，建议 50%。另外一点需要注意的是，在板级接口中实现 dev_wakeup_host_gpio 中断函数并注册，中断函数中需要调用 comm_driver 模块的 nfc_wakeup_host_gpio_isr 函数。具体可以参考已有的代码实现如表 2-4 所示。

表2-4 board_xxx.c 接口函数说明

函数	功能说明
int board_init(void *param)	板级初始化，主要是 GPIO、数据结构等的初始化。
void board_deinit(void)	NFC 关闭时的去初始化。
void board_pwn_ctrl(nfc_enable_ctrl_t enable)	控制 NFC 芯片上下电。
void board_wkup_host_gpio_int_enable(nfc_enable_ctrl_t en)	dev_wakeup_host_gpio 中断使能控制。
int board_get_dev_wkup_host_status(void)	获取 dev_wakeup_host_gpio 状态。
int board_get_host_wkup_dev_status(void)	获取 host_wakeup_dev_gpio 状态。
void board_set_host_wkup_dev_value(int value)	设置 host_wakeup_dev_gpio 状态。
const char *board_get_fw_cfg_path(void)	获取固件文件存放路径。
int board_i2c_write(char *buf, int count)	通过 I2C 往 NFC 芯片写数据（默认地址 0x28）。
int board_i2c_read(char *buf, int count)	通过 I2C 从 NFC 芯片读数据（默认地址 0x28）。

2.3.4 安全函数库编译指导

1. 参照 [2.2 获取驱动源码](#) 获取安全函数库源码。
2. 将如下文件添加到项目中，参加编译。

源文件	文件说明
include/securec.h	安全函数对外接口头文件。
include/securectype.h	对外接口辅助头文件，被 securec.h 包含。
src/memcpy_s.c	提供 memcpy_s 函数。
src/memmove_s.c	提供 memmove_s 函数。
src/memset_s.c	提供 memset_s 函数。
src/strcpy_s.c	提供 strcpy_s 函数。
src/strncpy_s.c	提供 strncpy_s 函数。
src/strcat_s.c	提供 strcat_s 函数。
src/strncat_s.c	提供 strncat_s 函数。
src/secureutil.c	提供获取安全函数版本号的函数。
src/snprintf_s.c	提供 snprintf_s 和 snprintf_truncated_s 函数。
src/vsnprintf_s.c	提供 vsnprintf_s 和 vsnprintf_truncated_s 函数。
src/sprintf_s.c	提供 sprintf_s 函数。
src/vsprintf_s.c	提供 vsprintf_s 函数。
src/secureprintoutput_a.c(包含 output.inl)	该文件为格式化输出公共函数，不提供浮点数以及宽字符处理功能。
src/sscanf_s.c	提供 sscanf_s 函数。
src/vsscanf_s.c	提供 vsscanf_s 函数。
src/secureinput_a.c(包含 input.inl)	该文件为格式化输入公共函数，不提供浮点数以及宽字符处理功能。
src/strtok_s.c	提供 strtok_s 函数。

2.4 编译适配

编译时可以适配驱动根目录的 makefile 文件，也可以不用驱动自带的 makefile，由上层 makefile 添加要编译的源文件和头文件。

- makefile 需要包含的源文件：comm_driver 目录、adapter/product/xxx 目录、第三方库 libboundscheck 目录。
- makefile 需要包含的头文件：comm_driver 目录、adapter/product/xxx 目录、adapter/oal/liteos 目录（liteos 根据自己命名的实际目录名确认）、第三方库 libboundscheck 的 include 目录。

此外，给芯片加载的固件可通过大数组方式获取（涉及固件包中 fw_array_file 文件夹下文件 fw_cfg.bin, fw_cfg.c, fw_ram.bin, fw_ram.c），或者文件读取方式获取（涉及固件包中 clockxxM 文件夹（xx 代替时钟频率）文件 nfc_cfg, RAM.bin）。通过编译宏 BUILD_FIRMWARE_TO_ARRAY 可进行选择，开启则采用大数组方式存储固件，否则采用文件方式存储固件。

2.5 nfc_dev.h 上层访问驱动接口

上层访问驱动的接口都在 nfc_dev.h 中定义，上层代码只需要包含这个头文件，即可访问驱动的所有功能，建议不要调用驱动其它文件的接口。当前定义了五个接口，实现一般不需要改动，如有需要，用户也可以定制。NFC 的驱动接口原则上只能由 NFC HAL 层代码调用。

表2-5 nfc_dev.h 接口函数说明

函数	功能说明
int hw_nfc_open(void)	打开 NFC，驱动初始化，NFC 芯片还未启动，此函数调用后，建议调用 hw_nfc_ioctl(1)，NFC 芯片上电。
int hw_nfc_close(void)	关闭 NFC，驱动去初始化，此函数调用前，建议调用 hw_nfc_ioctl(0)，NFC 芯片下电。
int hw_nfc_ioctl(int arg)	控制 NFC 芯片上下电。
int hw_nfc_write	往 NFC 芯片写数据。
int hw_nfc_read	从 NFC 芯片读数据，如果没有数据会阻塞。

3 NFC 软件移植

本章节将分步骤讲解，如何利用提供的 NFC 软件包，把 NFC 软件业务代码移植适配到 RTOS 嵌入式系统。

3.1 步骤一：获取 NFC 安装包

3.2 步骤二：定制产品的 HAL 层接口函数

3.3 步骤三：把 HAL 层加入编译流程

3.4 步骤四：把 Ndef 协议操作函数加入编译流程

3.5 步骤五：添加编译宏

3.6 步骤六：烧录版本

3.7 步骤七：测试 NFC 功能

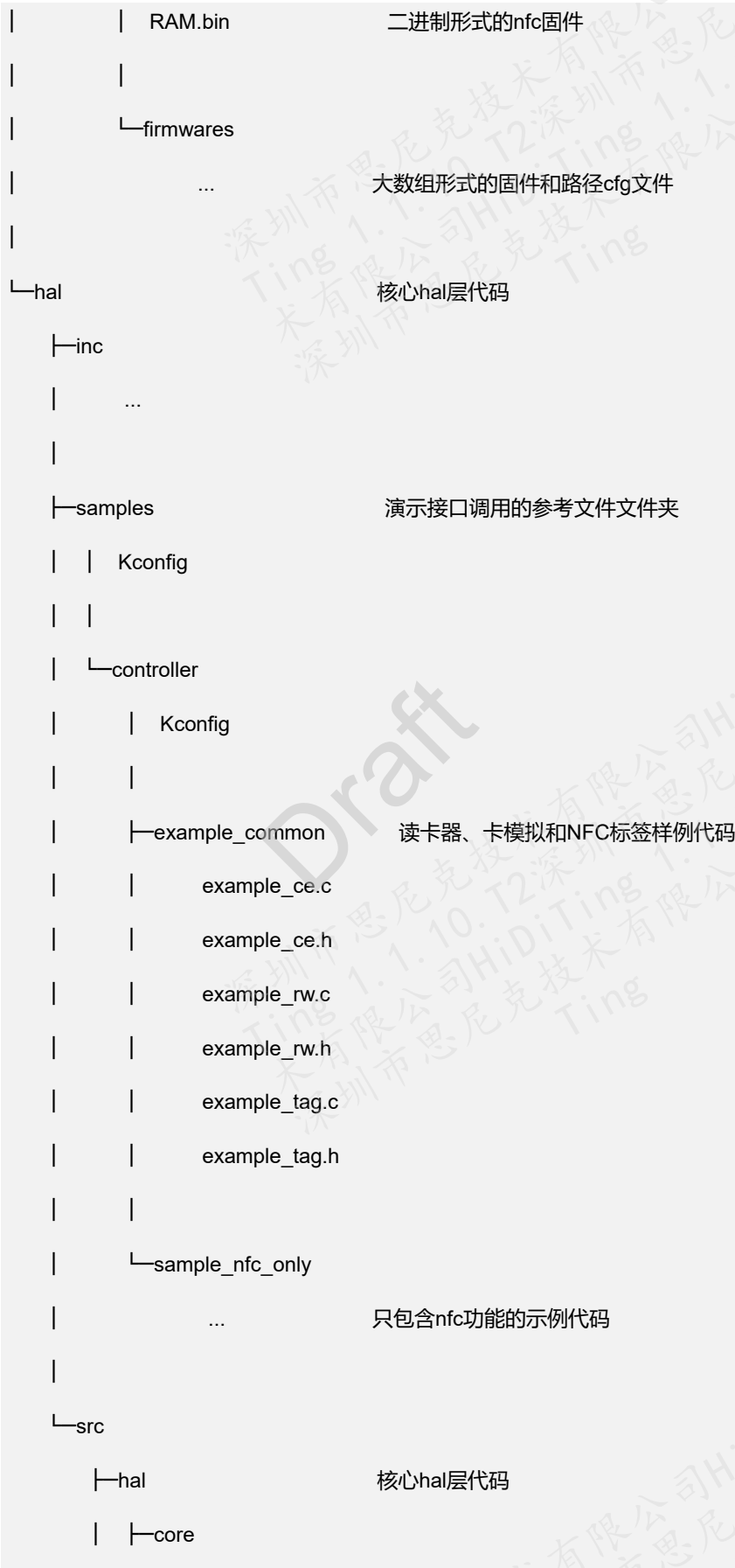
3.1 步骤一：获取 NFC 安装包

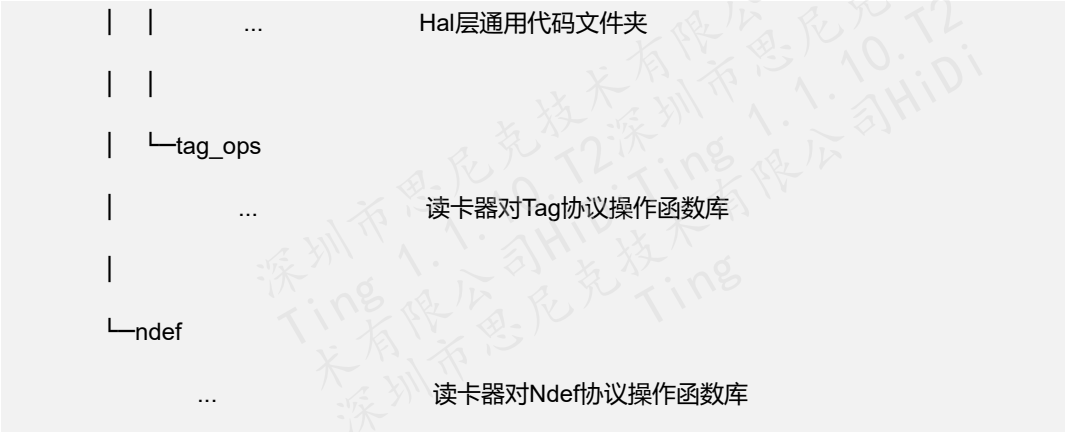
获取 NFC 安装包，并解压到 PC 端 Linux 目录，如路径 “\$(Linux_root)/vendor/nfc”。

解压后可获得 NFC HAL 层相关代码文件如下（hal 文件下为核心 hal 层代码，firmware 文件夹下为固件文件）：

```
$(Linux_root)/vendor/nfc.
```

```
├─firmware                    nfc固件
|
|   └─$(product_name)
|
|       └─27_12m
|
|           | nfc_cfg          二进制形式的config文件
```





关于 HAL 层 API 介绍，请参见相关的 NFC RTOS SDK API 文档。同时可以参考 sample 文件的实现内容，了解 HAL 层接口函数的调用方法。

3.2 步骤二：定制产品的 HAL 层接口函数

“\$(Linux_root)/vendor/nfc/hal/samples/controller/example_common/” 下面有 example_rw.c、example_ce.c 和 example_tag.c 三个功能示例代码，用户可以参照示例代码中 NFC_EXAMPLE_RwInit、NFC_EXAMPLE_CeInit 和 NFC_EXAMPLE_TagInit 展示的流程，根据产品进行参数或代码微调，需要调整的地方包括但不限于表 3-1 所示内容。

表3-1 HAL 层定制化参数修改

函数	描述	RW 模式	CE 模式	TAG 模式
NFC_HAL_SetCoreParams	设置 HAL 层核心参数	NA	默认配置： 1. NCI 控制包读操作超时时间 1000ms。 2. NCI 数据包读操作超时时间 1500ms。 3. 处理 NFCEE-HCI 消息的回调函	NA

函数	描述	RW 模式	CE 模式	TAG 模式
			数。	
NFC_HAL_NciSetConfig	配置 CORE_SET_CONFIG_CMD 命令	默认配置： 配置参数 1:0xA003，关闭状态保护定时器。 配置参数 2:轮询周期内 listen 时间 0x1F4 = 500ms。 配置参数 3:NFCC_CONFIG_CONTROL，允许 NFCC 整合 RF 参数。 配置参数 4:CON_DISCOVERY_PARAM，允许 Poll 模式。	默认配置： 配置参数 1:0xA300，nfcee enable bit，默认使能 eSE，产品根据芯片类型和使用场景来配置。	默认配置： 配置参数 1:0x80，使能场检测上报通知。
NFC_HAL_CfgLowPwr	配置 device 低功耗模式	默认开启低功耗模式。	默认开启低功耗模式	默认开启低功耗模式。
NFC_HAL_NciDiscoveryMap	配置 RF_DISCOVER_MAP_CMD 命令	默认配置： IsoDep 协议映射到 IsoDep 接口。	默认配置： IsoDep 协议映射到 IsoDep 接口。	NA
NFC_HAL_StartDiscovery	配置 RF_DISCOVER_CMD 命令	默认配置： 打开 ABFV 四种技术。	默认配置： 打开 A 技术。	默认配置： 打开 A 技术 (Tag 模式默认只支持 A 技术的 T2T)

3.3 步骤三：把 HAL 层加入编译流程

将 “\$(Linux_root)/vendor/nfc/hal/src/hal/” 文件夹下 HAL 层代码、
“\$(Linux_root)/vendor/nfc/hal/inc/” 文件夹下所需头文件和内核驱动代码统一放入产品工程路径下，修改相关的 Makefile，将 HAL 层所有源文件加入到编译主目标的流程中，并可访问到驱动的头文件。上层 NFC 相关 App 需要调用 HAL 层提供的接口函数，来实现整个 NFC 业务的流程，并编译最终目标。

请注意，若采用大数组方式加载固件，需要将固件数组文件（ca01_cfg.bin，ca01_fw_file_array.c，ca01_fw_file_array.h，ca01_fw0.bin）加入编译流程。

请注意，HAL 层若需要 RW/CE 功能，则需在用户 makefile 中分别使能编译宏 HAVE_NFC_POLL、HAVE_NFC_LISTEN。

3.4 步骤四：把 Ndef 协议操作函数加入编译流程

如果 RW 模式下需要添加 Ndef 协议操作函数库，则将
“\$(Linux_root)/vendor/nfc/src/ndef/” 文件夹下所有源文件、
“\$(Linux_root)/vendor/nfc/hal/inc/” 文件夹下对应头文件统一放入产品工程路径下，修改相关的 Makefile。上层 NFC 相关 App 需要调用 Ndef 协议操作函数库提供的接口函数，来实现 Ndef 协议操作的流程，并编译最终目标。

3.5 步骤五：添加编译宏

编译 RTOS 模块时根据需要添加下表中编译宏，请按照表 3-2 描述和需求自行添加。

表3-2 RTOS 编译宏

编译宏	功能
HAVE_NFC_POLL	NFC 开启读写器功能
HAVE_NFC_LISTEN	NFC 开启卡模拟功能

3.6 步骤六：烧录版本

将编译输出的结果烧录到产品中并测试系统是否加载正常。请注意，若采用读取文件方式加载固件，安装包内的 NFC 固件相关文件（RAM.bin 和 nfc_cfg）需要统一拷贝到目标机器的 “/odm/etc/firmware” 路径下。

请注意，这两个文件需要提前保存在 host 端，占用存储空间一般不超过 150KB，具体视用户所得固件文件大小情况而定。另外，默认拷贝路径可以通过搜索和修改平台驱动代码里的默认路径、以及 nfc_cfg 里的默认路径达成。

3.7 步骤七：测试 NFC 功能

运行用户 NFC 相关的 App，测试 NFC 业务是否可以正常运行。

4 常见问题

4.1 固件加载失败

4.2 NFCEE 激活失败

4.3 I2C 通路异常

4.1 固件加载失败

日志中出现如下任意一行信息，表明固件加载有问题，NFC 控制器的固件没有加载成功。请查看固件 FW 存放位置以及文件名和内容是否正确。

固件加载有问题时出现的信息如下：

- download fw firmware_get_cfg fail
- download firmware fail

4.2 NFCEE 激活失败

NFCEE (UICC) 的激活失败可能由如下原因导致：

- SWP 的 S1 电压存在抖动，会影响 NFC 控制器解析 SIM 卡响应的数据，优先排查 SWP 的硬件器件。
- 部分 SIM 卡除了 SWP 的接口外，还需要 Host 对 7816 接口激活，NFCEE 才能激活成功。

4.3 I2C 通路异常

I2C 通路异常通常为如下原因：

- 焊接异常，版本可以下载成功，业务初始化出现 Reset 消息失败。
- Host 为单核，处理 I2C 消息的优先级比较低，导致 I2C 数据未能及时被 Host 读取，NFC 无法释放业务资源，会产生硬复位，Host 需要对 NFC 重新进行初始化。

A 缩略语

C		
CE	Card Emulation	卡模拟
H		
HAL	Hardware Abstract Layer	硬件抽象层
N		
NFC	Near Field Communication	近场通信
NCI	NFC Controller Interface	NFC 控制器接口
R		
RW	Reader/Writer	读写器